

Monomorphization: Devirtualizing Java Call Sites via Static Analysis

Vansil R Chauhan (24M0847)
Samarth Tandale (24M0753)

Team “Dispatch Slayers” — CS 6004: Code Optimization for Object-Oriented Languages

April 19, 2026

Overview

Every instance method call in Java that is not statically resolvable – `a.speak()` on a variable declared as some supertype – compiles to `invokevirtual` (or `invokeinterface` when the static type is an interface). At runtime, the JVM resolves the actual target by walking the receiver object’s virtual method table (or interface method table). This indirection is exactly what makes polymorphism work, but it is paid for on *every* call, whether or not the call site is ever actually polymorphic in practice.

A call site is **monomorphic** if, given the whole program’s class hierarchy and the way its receiver is actually constructed and passed around, it can only ever resolve to a single concrete method – even though it is written using virtual dispatch syntax. At such a site the vtable/itable indirection buys nothing: every possible execution reaches the same target. If a compiler (static or JIT) can *prove* this, the call can be rewritten as a direct call, eliminating the dispatch.

This project, submitted as Programming Assignment 4 for CS 6004, builds exactly that: a static analyzer and Soot/Jimple bytecode transformer that identifies monomorphic call sites in an arbitrary input Java application and rewrites them from virtual dispatch to direct static calls. The critical word is *proves*: an unsound devirtualization silently changes program behavior. The tool is built so that it may fail to recognize a genuinely monomorphic site (a missed optimization), but it must never misclassify a polymorphic site as monomorphic (a correctness bug). Every design decision in the analysis follows from that constraint.

The tool ships two independently implemented analyses that answer the monomorphism question from different angles – a fast, call-graph-based check, and a much more expensive, hand-built interprocedural points-to analysis – plus a two-phase bytecode transformation, a ten-case correctness suite, and a dedicated microbenchmark used to obtain an honest, isolated measurement of the dispatch-cost saving. This report covers all of it in depth, grounded directly in the implementation under `submission/src/main/java/devirtualizer/`.

System Architecture

The tool is a command-line driver around Soot, the Java/Android bytecode analysis and transformation framework. Soot loads an application’s class files into its own intermediate representation,

Jimple – a three-address-code form of bytecode that is far easier to pattern-match and rewrite than raw class-file bytecode – builds a whole-program call graph, and exposes a pluggable pack/phase system for whole-program transformations.

`Main.java` parses two required arguments (application directory, main class fully-qualified name) and one optional analysis level, builds the Soot command line (`-w` for whole-program mode, `-process-dir` to pull in every class under the application directory, `-f c` to emit `.class` output), and registers three custom transformers into Soot’s `wjtp` (whole-jimple-transformation-pack) pipeline:

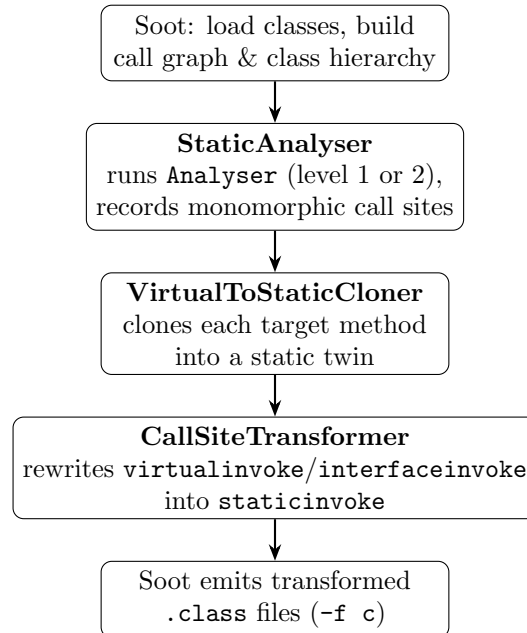
Listing 1: Pipeline registration in `Main.java`

```

1 Pack pack = PackManager.v().getPack("wjtp");
2 pack.add(new Transform("wjtp.staticAnalyser",
3     new StaticAnalyser(analyser, state)));
4 pack.add(new Transform("wjtp.virtualToStaticCloner",
5     new VirtualToStaticCloner(state)));
6 pack.add(new Transform("wjtp.callSiteTransformer",
7     new CallSiteTransformer(state)));
8 soot.Main.main(sootArgs);

```

Soot guarantees these run in registration order after it has built the whole program’s call graph and class hierarchy, which is exactly the ordering the tool depends on: analysis must complete before cloning can know which methods need static twins, and cloning must complete before call sites can be rewritten to invoke those twins.



Shared mutable state between the three phases lives in a single `OptimizationState` object: a `Map<Unit, SootMethod>` from call-site unit to its proven monomorphic target, a `Map<SootMethod, SootMethod>` from an original virtual method to its generated static twin, and a running count of total call sites (used only for the summary line `Main` prints at the end – “Callsites transformed m/n ”). Routing all cross-phase communication through one small, explicit state object rather than, say, having later phases re-derive information from Soot’s scene, keeps the three transformer classes decoupled and independently testable in principle.

`ApplicationArguments` is a minimal hand-rolled flag parser (`-d/--directory`, `-c/--main-fqnc`, `-l/--level`) with no external dependency – adequate for a two-flag-plus-one CLI tool, though it silently tolerates a stray non-flag first token due to an empty conditional body at line 29-30 of `ApplicationArguments.java` (a harmless leftover, since every subsequent branch of the `switch` still requires a recognized flag).

Analysis 1: Call-Graph-Based Monomorphism

`CallGraphBasedAnalysis` is the cheap of the two analyses, and it leans entirely on Soot’s own call graph construction (Class Hierarchy Analysis by default, when no more precise algorithm is configured). For every concrete method’s body, for every `VirtualInvokeExpr` unit, it asks the call graph for the outgoing edges from that unit and collects the concrete targets:

Listing 2: Monomorphism check via call-graph out-edges

```

1 Set<SootMethod> targets = new HashSet<>();
2 Iterator<Edge> edges = callGraph.edgesOutOf(unit);
3 while (edges.hasNext()) {
4     SootMethod target = edges.next().tgt();
5     if (target != null && target.isConcrete()) {
6         targets.add(target);
7         if (targets.size() > 1) break;    // early exit
8     }
9 }
10 if (targets.size() == 1) {
11     monomorphicCallSites.put(unit, targets.iterator().next());
12 }

```

The early exit on the second distinct target is a small but deliberate efficiency: the analysis only needs to distinguish “exactly one” from “more than one,” so it never needs to enumerate a large target set fully.

Two properties of this analysis are worth being explicit about. First, its precision is entirely inherited from Soot’s call graph – it never inspects points-to information itself, so any imprecision in CHA (which resolves a call site to *every* override in the class hierarchy reachable from the static type, regardless of whether that subtype is ever actually instantiated) is imprecision the tool cannot recover from at this level. Second, as implemented, it only matches `VirtualInvokeExpr` – it does not special-case `InterfaceInvokeExpr`. Since interface calls also appear in Soot’s call graph with the same edge structure, an interface call site with a single call-graph target is simply never considered by this particular analysis pass (it would need the `instanceof` check loosened to `InstanceInvokeExpr`, as the second analysis does). This is a real, narrower-than-documented scope for level 1 specifically – Section 6 verifies interface dispatch (`Test2`) is nonetheless monomorphized correctly, but only because level 2 (points-to-based) is the one used to devirtualize it in practice.

Analysis 2: Inter-Procedural Points-To Analysis

`InterPTABasedAnalyser` is the substantive contribution of the project: rather than trusting a pre-built call graph, it runs a hand-implemented, **inter-procedural**, **context-sensitive**, **flow-sensitive** points-to analysis starting from the program’s `main` method, and asks – for every instance call site reached during that traversal – what set of concrete allocation sites the receiver can actually hold. If that set maps to exactly one target method, the site is monomorphic.

All three qualifiers are load-bearing for precision, and each is worth spelling out because each fixes a specific, concrete imprecision the others don't:

- **Flow-sensitive** – the analysis respects statement order within a method body. A local reassigned between two calls (`d = new Dog(); d.speak(); d = new Cat(); d.speak();`) has a different points-to set at each call, which a flow-*insensitive* analysis (one global set per variable) would conflate into `{Dog, Cat}` at both sites, destroying monomorphism at both.
- **Context-sensitive** (call-strings approach) – the analysis distinguishes different call sites into the same method rather than merging their effects. A helper called once with a `Dog` receiver and once with a `Cat` receiver would, under a context-*insensitive* analysis, appear to see `{Dog, Cat}` at *both* call sites once their effects are merged back into one shared summary for the helper. Call-string sensitivity keeps each call's analysis instance separate.
- **Inter-procedural** – points-to information is threaded across method boundaries in both directions: a receiver's possible identity is frequently determined by what its caller passed in (context flowing down), and the fields of a heap object may be mutated inside a callee (heap effects flowing back up).

Abstract object domain

The analysis tracks not concrete objects but a small closed lattice of *abstract* objects, defined in PTAToolkit/:

Class	Represents
<code>RealObject</code>	An allocation site (<code>new</code> expression), identified by <i>source line number</i> – the analysis's abstraction of “one allocation site = one abstract object,” not one object per run.
<code>ParameterObject</code>	The unknown value bound to a formal parameter; distinct per <code>ParameterRef</code> , so parameter 0 and parameter 1 of the same method never alias by construction.
<code>ThisObject</code>	The receiver of the enclosing method (bound to whatever points-to set the caller passed at the call site being analyzed).
<code>NullObject</code>	The literal <code>null</code> value; a singleton.
<code>UndefinedObject</code>	The initial (bottom-ish) value every local starts at, before any assignment reaches it in the flow analysis.

`ParameterObject` and `ThisObject` are grouped under an abstract `DummyObject` (objects that stand in for a value whose concrete identity is not known at this analysis scope, as opposed to `RealObject`, which *is* a concrete allocation the analysis has actually witnessed). `NullObject` and `UndefinedObject` sit directly under `AbstractObject`, the analysis's overall value domain – they are not “heap objects” (they have no fields, no `SootClass`) but must still be legal elements of a points-to set. This is why `PointsToSet` internally stores `Set<AbstractObject>` and only filters down to `HeapObject` instances (`RealObject` $\cup$ `DummyObject`) when a caller specifically needs objects with class identity – e.g. to resolve which method a call dispatches to.

Distinguishing `ParameterObject/ThisObject` from “unknown” or from nothing at all is what keeps the analysis sound at method boundaries: a parameter's contents genuinely are not known within the callee in isolation, so representing that explicitly (rather than, say, treating an unresolved local as an empty points-to set) prevents the analysis from silently concluding monomorphism it hasn't

actually earned. An empty set would look identical to “this variable can never hold anything,” which is a much stronger and generally false claim.

The allocation-site abstraction (one abstract object per `new` statement, keyed by line number, not per dynamic instance) means the analysis is not object-sensitive in the fuller sense some points-to literature uses the term – two `Dog` instances created by two *different* loop iterations of the same `new Dog()` statement are indistinguishable to the analysis; they collapse to the same `RealObject`. This is a standard and reasonable precision/cost tradeoff (finer per-instance tracking would require unbounded state), and it does not threaten soundness – it can only ever cause two objects to be merged that a finer analysis would have kept apart, which can only lose precision (fail to prove monomorphism), never gain unsound precision.

Program state: stack, heap, and escape set

`PointsToState` is the dataflow lattice element the `ForwardFlowAnalysis` propagates through the CFG. It has three parts:

- **Stack** – `Map<Local, PointsToSet>`, what each local variable currently points to.
- **Heap** – `Map<HeapObject, Map<ObjectField, PointsToSet>>`, field-sensitive: what each field of each known heap object currently points to. `ObjectField` identifies a field by (declaring-class-name, field-name) pair, so fields declared in different classes are never conflated even if same-named.
- **Escape set** – `Set<HeapObject>`, objects that are reachable from outside the current analysis scope (see below).

`merge` at CFG join points does what a standard may-analysis does: union stacks, union heap field sets, union escape sets (`PointsToState.java`, lines 72-81). Field updates on a store, however, are more careful than a blind union – this is where **strong vs. weak update** comes in. When a field store’s base local resolves to a points-to set of size *exactly one* concrete heap object, the store can safely *replace* that field’s set (a strong update, since the analysis knows precisely which object is being written). When the base could be more than one object, the store can only ever *add* to each candidate’s field set (a weak update), because in some execution the store might hit a different one of those objects, leaving the others’ old value intact:

Listing 3: Strong vs. weak update on field store (`PointsToAnalyser.handleFieldStore`)

```

1 Set<HeapObject> pts_base_set = pts_base.getHeapObjects();
2 if (pts_base_set.size() == 1) { // strong update
3     HeapObject target = pts_base_set.iterator().next();
4     out.strongUpdate(target, field, resultSet);
5 } else { // weak update
6     for (HeapObject obj : pts_base_set) {
7         out.weakUpdate(obj, field, resultSet);
8     }
9 }

```

Escape analysis: why static field stores need special care

A points-to analysis scoped to one call chain has no visibility into what happens to an object once a reference to it is stored somewhere the analysis doesn’t track – most importantly, a **static** field, which is reachable from anywhere in the whole program at any point after the store.

`PointsToState.escape` handles this: when an object (or anything transitively reachable from it through its own fields) is stored into a static field, the whole reachable closure is added to the **escape set** via a breadth-first traversal over the heap graph the analysis already has:

Listing 4: Escape propagation (`PointsToState.escape`)

```

1 public void escape(PointsToSet rootPointsToSet, PointsToState in) {
2     Queue<HeapObject> que = new ArrayDeque<>(rootPointsToSet.getHeapObjects());
3     escapingSet.addAll(rootPointsToSet.getHeapObjects());
4     while (!que.isEmpty()) {
5         HeapObject obj = que.poll();
6         for (ObjectField sf : Util.getFields(obj.getSootClass())) {
7             for (HeapObject next : in.getPointeeSet(obj, sf).getHeapObjects())
8                 {
9                     if (escapingSet.contains(next)) continue;
10                    escapingSet.add(next);
11                    que.add(next);
12                }
13         }
14     }
15 }

```

The escape set matters directly for field stores through a static field *read*: a load from a static field (`handleFieldLoad`, lines 358-364 of `PointsToAnalyser.java`) doesn’t know which of possibly many writers throughout the program most recently wrote it, so it conservatively returns the *entire escape set* as the load’s points-to set, rather than trying to track a precise value for globally-visible state. This is the analysis choosing soundness over precision at a boundary it cannot see across – exactly the same philosophy as treating parameters as a distinct, opaque `DummyObject` rather than guessing.

Call-string context: threading state across method boundaries

Each `PointsToAnalyser` instance analyzes exactly one method body under one specific calling context. The context is threaded in through the constructor as the receiver’s points-to set, each argument’s points-to set, the caller’s heap/escape state at the call site, and the accumulated **call stack** – a `List<Pair<Integer, SootMethod>>` of (source line, target method) pairs recording the chain of calls that led here (this is literally the “call-strings” context-sensitivity approach). When a call site is reached inside `handleInvokeExpr`, the analysis:

1. Resolves the receiver’s current points-to set from the state *at that program point* (flow-sensitivity feeding context-sensitivity).
2. Computes each argument’s points-to set the same way.
3. Calls `getFeasibleTargets` to resolve, for each concrete object the receiver might be, which method it dispatches to (see below).
4. Recursively constructs a *new* `PointsToAnalyser` for each feasible target, seeded with that call’s specific receiver/argument context and an extended call stack – this is the actual inter-procedural, context-sensitive traversal.
5. Merges the return-flow effects (heap mutations visible at each target’s **return** statements) of every feasible target back into the caller’s outgoing state.

Listing 5: Recursive, context-threaded call into a target (`handleInvokeExpr`)

```

1 for (SootMethod target : feasibleTargets) {
2     if (target.isJavaLibraryMethod()) continue;
3     Body body = target.retrieveActiveBody();
4     if (body == null) continue;
5
6     List<Pair<Integer, SootMethod>> newCallStack =
7         new ArrayList<>(callStack);
8     newCallStack.add(new Pair<>(u.getJavaSourceStartLineNumber(), target));
9     PointsToAnalyser pta =
10        new PointsToAnalyser(body, thisPTS, paramPTS, in, newCallStack);
11     callSitePTA.get(u).put(target, pta);
12 }

```

Each recursive `PointsToAnalyser` construction immediately runs its own fixed-point `doAnalysis()` to completion inside the constructor (line 90 of `PointsToAnalyser.java`) – the analysis is built as a strict, eagerly-recursive descent through the call tree from `main`, not a worklist over the whole program. Every `PointsToAnalyser` instance it creates along the way is cached per (call-site, target) pair in `callSitePTA`, which is exactly what `InterPTABasedAnalyser` later walks to build its call-site-to-target-set map (Section 4.6).

Feasible target resolution

`getFeasibleTargets` turns a points-to set of candidate receiver objects into a set of concrete target methods, using Soot’s class hierarchy directly rather than the call graph:

Listing 6: Target resolution per candidate receiver object

```

1 for (HeapObject obj : basePointsTo.getHeapObjects()) {
2     SootClass receiverClass = obj.getSootClass();
3     if (receiverClass == null) continue;
4     if (expr instanceof InterfaceInvokeExpr
5         && receiverClass.isInterface()) continue;
6
7     SootMethod target =
8         hierarchy.resolveConcreteDispatch(receiverClass, expr.getMethod());
9     if (target != null && target.isConcrete())
10        targets.add(target);
11 }

```

`Hierarchy.resolveConcreteDispatch` performs exactly the virtual dispatch resolution the JVM itself would do at runtime for a receiver of that concrete class – it walks up from `receiverClass` to find the most-derived override of the invoked signature. Because the input to this resolution is the analysis’s own (precise, context- and flow-sensitive) points-to set rather than the static declared type the call graph would use, this is exactly where the points-to analysis can out-precision the call-graph-based analysis: it may know the receiver can *only* be a `Dog`, even though the call graph (built from CHA over the static type `Animal`) would have to consider every override of `Animal.speak()` in the whole class hierarchy as a possible target.

`StaticInvokeExpr` and `SpecialInvokeExpr` (constructors and super-calls) resolve trivially to their single declared method – there is no dispatch ambiguity to resolve for either, which is also why both are explicitly excluded from monomorphism accounting entirely (there is nothing to devirtualize about a call that was never virtual).

From per-method analysis to a monomorphic call-site set

`InterPTABasedAnalyser.performAnalysis` drives the whole thing: it runs one top-level `PointsToAnalyser` over `main`, then recursively walks the `callSitePTA` cache built during that analysis (`getCallSiteToTargetMapping`) to build a flat `Map<Unit, Set<SootMethod>>` over every reachable instance call site, unioning target sets contributed by every context the site was reached under. A call site lands in `monomorphicCallSites` only if that union has exactly one element – i.e. it resolved to the same single target *across every calling context* the analysis explored, which is the correct, sound criterion (a site that is monomorphic under one caller but polymorphic under another is, correctly, not global monomorphism).

This recursive walk has no cycle detection. That is the direct, mechanical source of the tool’s one documented hard limitation: **the input program’s call graph must be acyclic** (no direct or mutual recursion), because the call-string context has no bounded depth k – an unbounded call-string approach applied to a cyclic call graph would recurse forever (both in the eager `doAnalysis()` construction chain in Section 4.4, and in this traversal). This is recorded explicitly in the original submission report rather than silently failing, and is revisited as the top future-work item in Section 9.

Transformation: From Analysis to Rewritten Bytecode

Knowing a call site is monomorphic is not enough to devirtualize it in place: a `staticinvoke` instruction has no implicit receiver, so the target method needs a static counterpart to call. The tool splits this into two phases, each its own `SceneTransformer` in the `wjtp` pipeline.

Phase 1: VirtualToStaticCloner

For every distinct method that is the target of at least one monomorphic call site, `VirtualToStaticCloner.clone` builds a static twin named `<original>_static`, with the receiver’s declaring class appended as one extra trailing parameter. Critically, **the original method is left untouched** – other, non-monomorphic call sites into the same method still need to dispatch to it virtually, and deleting it would be unsound for any caller the analysis didn’t (or couldn’t) prove monomorphic. The clone is purely additive.

Building the clone body is a careful three-part copy over Jimple’s representation, because Jimple statements and their operands are mutable graph nodes that cannot simply be shared between two method bodies:

1. **Locals** are cloned one-for-one into a fresh `Map<Local, Local>`, so every reference inside the new body can be redirected to a body-local copy rather than aliasing the original method’s locals.
2. **Units (statements)** are cloned via Soot’s `Unit.clone()`, *except* for the identity statement that binds `@this` – that one is rebuilt from scratch as a `ParameterRef` at the new trailing parameter index, since a static method has no `ThisRef` at all:

Listing 7: Rewriting the `@this` binding into a trailing parameter

```
1  if (u instanceof IdentityStmt
2      && ((IdentityStmt) u).getRightOp() instanceof ThisRef) {
3      newUnit = Jimple.v().newIdentityStmt(
4          localMap.get(id_stmt.getLeftOp()),
```



```

5         Jimple.v().newParameterRef(
6             declaringClass.getType(), vm.getParameterCount());
7     } else {
8         newUnit = (Unit) u.clone();
9     }

```

Every other cloned unit then has its use/def value boxes walked and any `Local` operand redirected through `localMap`, so the clone refers only to its own locals.

3. **Branch targets** are fixed up in a second pass over the fully cloned unit list: any `UnitBox` (the indirection Jimple uses for jump targets, so that a `goto` or `if` can be retargeted without invalidating other references) pointing at an old unit is redirected to that unit's clone via a parallel `Map<Unit, Unit>`. This two-pass structure (clone everything first, then fix up cross-references) is necessary because a branch can target a unit that hasn't been cloned yet when the branching statement itself is processed.

`isValid` guards the clone against method shapes that cannot be soundly turned into a static twin this way: already-static, abstract, native, or synchronized methods, or ones with no retrievable body, are all rejected up front (with a diagnostic to `stderr`) rather than silently producing a broken clone.

Phase 2: CallSiteTransformer

For every recorded monomorphic call site whose target has a static twin, `transformCallSite` performs the actual rewrite. It extracts the existing invoke expression (from either an `InvokeStmt` or the right-hand side of an `AssignStmt`, covering both a bare call and a call whose result is used), appends the receiver as an additional trailing argument, and constructs a `staticinvoke` expression against the cloned method:

Listing 8: Call-site rewrite (`CallSiteTransformer.transformCallSite`)

```

1 List<Value> newArgs = new ArrayList<>(iie.getArgs());
2 newArgs.add(iie.getBase());
3 InvokeExpr newExpr =
4     Jimple.v().newStaticInvokeExpr(staticMethod.makeRef(), newArgs);
5
6 if (s instanceof InvokeStmt) {
7     ((InvokeStmt) s).setInvokeExpr(newExpr);
8 } else if (s instanceof AssignStmt) {
9     ((AssignStmt) s).setRightOp(newExpr);
10 }

```

Worked example

The following is the tool's actual, verified output (from the original submission report) for the source pattern used in `Test6`: a call through a statically-typed `Animal` reference that can only ever hold a `Dog`.

Listing 9: Before

```

1 Animal d = new Dog();
2 d.speak();

```

Listing 10: After (level 2, points-to-based)

```
1 class Dog extends Animal {
2     public void speak() { System.out.println("Woof!"); }
3     public static void speak_static(Dog d) {
4         System.out.println("Woof!");
5     }
6 }
7 ...
8 Animal d = new Dog();
9 Dog.speak_static((Dog) d);
```

At the bytecode level this is a change from `invokevirtual Animal.speak:()V` to `invokestatic Dog.speak_static:(LDog;)V` – confirmed directly via `javap -c` on the transformed class file during evaluation, which is exactly the dispatch elimination the whole tool exists to produce.

Correctness: The Test Suite

Correctness is checked with ten hand-written Java programs (`tests/Test1.java–Test10.java`), each isolating one dispatch shape or edge case. Every test is compiled, transformed, and re-run; the transformed program’s output is compared byte-for-byte against the original’s. All ten pass.

Test	Dispatch shape under test
Test1	Three-level inheritance chain ($A \rightarrow B \rightarrow C$); receiver statically typed A but only ever holds a C .
Test2	<code>interfaceinvoke</code> dispatch through an interface reference (Speakable) with a single implementer (Dog).
Test3	Multiple distinct monomorphic call sites on the same receiver within one loop body (four different methods).
Test4	No inheritance at all – receiver’s static type already equals its only possible dynamic type.
Test5	Monomorphic call with a non-object argument (<code>int volume</code>), checking the parameter list is threaded through the clone correctly.
Test6	Two receivers of the <i>same</i> declared type but different dynamic behavior at two separate call sites – both prove independently monomorphic (used as the worked example in Section 5.3).
Test7	A monomorphic call whose target itself makes a further monomorphic call (<code>speak()</code> calls <code>helper()</code>) – exercises the analysis’s inter-procedural recursion one level deep.
Test8	Two subclasses with <i>textually identical</i> overridden method bodies – checks the analysis distinguishes them by declaring class, not by looking at whether their bytecode happens to match.
Test9	A monomorphic instance call mixed with an already-static call (<code>Animal.staticSpeak()</code>) in the same loop – checks the static call is correctly left alone (it was never virtual).
Test10	Two receivers of the same declared type where <i>one</i> is provably monomorphic (d , only ever a Dog) and the other is <i>not</i> (a , declared and allocated as the base Animal type, but in a hierarchy with siblings Dog / Cat) – a same-file positive/negative control pair.

Test10 is worth calling out specifically: it is the one test explicitly designed to confirm the analysis does *not* over-devirtualize. Its inline comments (`// monomorphized / // not monomorphized`) record the expected classification directly in the test source, functioning as a lightweight expected-output oracle alongside the runtime output-equality check.

Performance Evaluation

Why the correctness suite cannot measure performance

The obvious first instinct – time **Test1–Test10** before and after transformation, using the harness already built for exactly that (`script.sh`) – was tried and discarded as a measurement strategy. Each test completes in roughly 50–60 ms, almost entirely JVM startup cost, and issues at most a handful of virtual calls total. Repeated runs bounced between –22% and +16% swing between the “before” and “after” timings of the *same* unchanged binary – pure measurement noise, not signal. There is essentially no dispatch cost to remove in a program that barely dispatches.

A dedicated microbenchmark

perf/PerfBenchmark.java isolates dispatch cost directly: the same `Animal/Dog` monomorphic shape as `Test6`, but with the call issued 200,000,000 times in a tight loop, so cumulative per-call dispatch overhead rises far above JVM startup noise:

```
1 Animal d = new Dog();
2 long result = 0;
3 for (int i = 0; i < 200_000_000; i++) {
4     result += d.speak();
5 }
6 System.out.println(result);
```

Before trusting any timing number, two things were verified independently:

1. **Behavioral equivalence** – both the original and the transformed program print the same value, 400000000.
2. **The transformation actually happened at the bytecode level** – `javap -c` on the transformed class confirms the call site changed from `invokevirtual` to `invokestatic` `Dog.speak_static:(LDog;)I`.

The second check specifically guards against a timing difference that looks meaningful but is actually measuring nothing – e.g. a transformation pass that silently failed to apply would still produce *some* timing delta from run-to-run noise, and without confirming the bytecode actually changed, that delta could easily be misread as a real result.

Results

Five timed runs of each variant, executed with `java -Xint` (JIT compilation disabled – see below for why):

Variant	Range	Mean	Median
Original	23.5 – 25.1 s	24.36 s	24.38 s
Transformed	22.1 – 24.7 s	22.81 s	22.36 s
Improvement		~6.4%	~8.3%

Why -Xint, and what the number does and does not mean

This qualification is essential to reading the result honestly, and is the single most important methodological point in the whole evaluation.

Under a normal, JIT-enabled JVM, HotSpot performs its own runtime devirtualization via **inline caching and speculative devirtualization**: it observes that a call site has only ever seen one receiver type, and after a brief warmup, speculatively inlines that target behind a cheap type guard – achieving much the same effect as this project’s static transformation, and often *more*, because the JIT can exploit actual runtime type history that a static analysis has no access to. Run this same benchmark without `-Xint` and C2’s own devirtualization would likely erase most or all of the measured gap, because it is solving the same problem this tool solves, just at runtime and with strictly more information available.

So the honest claim the 6–8% number supports is *not* “this makes production Java 6–8% faster.” It is: *static devirtualization eliminates interpreter-mode dispatch cost, and removes the dependency*

on JIT warmup to obtain that benefit. Measuring under `-Xint` is the methodologically correct choice for the actual question under test – does static devirtualization remove dispatch overhead? – precisely because it isolates that effect from the JVM’s own competing, and largely overlapping, optimization.

This is also the direct answer to the natural follow-up question “then where does this optimization actually matter?” – Section 8 returns to it.

Design Decisions and Tradeoffs

Two independent analyses instead of one refined one. Rather than building a single maximally-precise analysis, the project implements a cheap CHA-call-graph-based check and an expensive hand-rolled points-to analysis side by side, selected at the command line via `-l 1 / -l 2`. This mirrors a real compiler engineering tradeoff: a fast, approximate pass that catches the easy cases cheaply, versus a slow, precise pass reserved for when the cheap one isn’t enough. The one gap the implementation leaves open, called out explicitly as future work in Section 9, is that the two were never empirically compared against each other on the same input – there is no measurement of how many additional sites (if any) the points-to analysis proves monomorphic that the call-graph analysis misses, which is the natural next experiment given both already exist.

A hand-rolled points-to analysis rather than Soot’s built-in Spark/Paddle. Soot ships its own configurable points-to analysis frameworks, which would have been the lower-effort path to a working analysis. Building `PTAToolkit` from scratch on top of Soot’s lower-level `ForwardFlowAnalysis` scaffolding instead is a deliberate choice to own the full precision story end-to-end – particularly context-sensitivity via explicit call strings and field-sensitive strong/weak update, which are exactly the properties Section 4 argues are load-bearing for proving monomorphism, and which are easier to guarantee by construction in a purpose-built analysis than to configure correctly in a general-purpose one.

Allocation-site abstraction, not full object-sensitivity. As discussed in Section 4.1, collapsing every dynamic instance from one `new` statement into a single abstract `RealObject` is a soundness-preserving precision/cost tradeoff. It is the same choice nearly every practical points-to analysis makes (k -object-sensitivity, when implemented, refines rather than replaces this baseline), and it keeps the abstract domain’s size bounded by program size rather than by execution length.

Cloning rather than mutating the original method. Keeping the original virtual method intact and adding a static twin, rather than converting the method in place, follows directly from soundness: any other call site into the same method that was *not* proven monomorphic still needs the original virtual dispatch to work correctly for whichever receiver type it actually sees at runtime. The cost is a small amount of code duplication (and a correspondingly larger `.class` file); the benefit is that the transformation never has to reason about “is this the *only* caller” before touching a method.

Unbounded call-strings, and its direct consequence: no recursion support. Choosing not to bound the call-string depth k keeps the context-sensitivity implementation simpler (no depth-cutoff merging logic, no loss of precision from collapsing contexts once k is exceeded), at the direct cost of the analysis being undefined – and, as implemented, actually diverging via unbounded recursion – on any input whose call graph has a cycle. This is documented as an explicit, known restriction on accepted input programs rather than an unnoticed bug, which is the right way to ship a limitation that would otherwise be a silent non-termination.

Limitations and Future Work

- **The two analyses have never been compared against each other.** Both are fully implemented and independently correct, but no experiment establishes how many monomorphic sites the points-to analysis identifies that the call-graph analysis misses (or, in principle, vice versa, though that direction is less likely given the points-to analysis strictly refines call-graph information). This is the single most valuable missing result in the project – it is the direct, measurable answer to “does the expensive analysis earn its cost?”
- **No recursion support.** The absence of a bounded call-string depth k restricts accepted input to acyclic call graphs (Section 4.6, Section 8). Implementing k -limiting – collapsing call strings beyond depth k into a merged, less-precise context – is the standard remedy and the natural next feature.
- **Benchmarked on one synthetic microbenchmark.** The 200-million-call loop isolates per-call dispatch cost precisely, which is exactly what it was built to do, but says nothing about aggregate benefit on a real application, where monomorphic call sites are typically a modest fraction of all call sites. Running the tool over a standard benchmark suite (e.g. DaCapo) would give a realistic, whole-program figure.
- **No JIT-enabled measurement.** The prediction in Section 6.4 that gains would largely vanish under a warmed C2 compiler is reasoned from how HotSpot’s inline caching works, not directly measured. Running the same benchmark without `-Xint` (ideally with warmup iterations discarded) would turn that prediction into a checked result.
- **Interface dispatch is implemented but only lightly tested.** `interfaceinvoke` rewriting works end-to-end (`Test2` exercises it, via level 2), but the ten-case suite otherwise concentrates on class-hierarchy dispatch; interface-heavy programs (e.g. multiple interfaces, default methods) are not separately covered.
- **Level-1 (call-graph) analysis silently under-covers interface calls.** As noted in Section 3, `CallGraphBasedAnalysis` only matches `VirtualInvokeExpr`, not `InterfaceInvokeExpr`. A program relying solely on level 1 for interface-heavy code would miss monomorphic interface call sites that level 2 would catch. Widening the `instanceof` check to `InstanceInvokeExpr` (matching what `InterPTABasedAnalyser` already does) is a small, low-risk fix.

Artifacts

Component	Location
Driver / CLI	Main.java, ApplicationArguments.java
Cross-phase shared state	OptimizationState.java
Call-graph-based analysis	analyser/CallGraphBasedAnalysis.java
Points-to-based analysis	analyser/InterPTABasedAnalyser.java
Points-to analysis engine	PTAToolkit/PointsToAnalyser.java (496 lines)
Points-to state & sets	PTAToolkit/PointsToState.java, PointsToSet.java
Abstract object domain	PTAToolkit/ {HeapObject, RealObject, ParameterObject, ThisObject, NullObject, UndefinedObject, DummyObject}.java
Method cloner	transformer/VirtualToStaticCloner.java
Call-site rewriter	transformer/CallSiteTransformer.java
Total-call-site counter	StaticAnalyser.java
Correctness suite	tests/Test1.java–Test10.java
Performance microbenchmark	perf/PerfBenchmark.java
Before/after timing harness	script.sh
Original graded submission report	submission/report.pdf